



Curso:	Disciplina:	Código/Turma:
Professor/a:	Data:	
Aluno/a:	Matrícula:	

Recomendações e regras para a avaliação

- Proibido qualquer **interação com outro estudante** durante o período da prova.
- O tempo de realização da prova será de no máximo 1h40m.
- Os alunos que terminarem a prova antes do final da aula, podem sair de sala após 30 minutos de duração.
- Implemente as questões em Java ou Python e envie o código no AVA

-
1. **Implementação de Dijkstra e execução em um grafo.** Implemente o algoritmo de **Dijkstra** para caminhos mínimos a partir de uma fonte única. **Não é permitido** usar funções de biblioteca que já forneçam caminho mínimo prontas (ex.: `networkx.shortest_path`, `boost::dijkstra_shortest_paths`, `Graphs.dijkstra` etc.).

Sua implementação deve receber como entrada o grafo abaixo e o vértice fonte $s = A$, e produzir como saída, para cada vértice v do grafo:

- o custo $d[v]$ do menor caminho de s até v ;
- a sequência de vértices do **caminho mínimo**, ex: $A -> B -> C -> D...$

Atenção: o grafo possui vértices que **não são alcançáveis** a partir de A . Sua implementação deve detectar esse caso e sinalizar explicitamente na saída — por exemplo, $d[v] = \infty$ e caminho vazio.

O que entregar:

- (a) O **código-fonte** completo da sua implementação de Dijkstra, comentado nos pontos relevantes.
- (b) O resultado da execução do código mostrando todos os menores caminho até a origem e a distancia até eles.

Grafo — dirigido, 20 vértices, fonte $s = A$. Lista de adjacência no formato vértice -> [(vizinho, peso), ...]:

```
A -> [(B, 4), (C, 2), (D, 7)]
B -> [(C, 1), (E, 3), (F, 6)]
C -> [(D, 4), (F, 8), (G, 5)]
D -> [(G, 2), (H, 3)]
E -> [(F, 2), (I, 5)]
F -> [(I, 1), (J, 4)]
G -> [(J, 3), (K, 7)]
H -> [(K, 4), (L, 6)]
I -> [(M, 3)]
J -> [(K, 2), (M, 4), (N, 6)]
K -> [(L, 3), (O, 8)]
L -> [(O, 2)]
M -> [(N, 1)]
N -> [(O, 4)]
O -> []
P -> [(Q, 3), (R, 9)]
Q -> [(R, 5)]
R -> [(S, 2)]
S -> [(T, 4)]
T -> [(P, 6)]
```